

MAL Form Monitor — Updated User Guide

Version 1.1 • Current program behavior

A practical guide for setup and everyday use

This guide keeps the presentation style of the original guide while updating the instructions for the current program. The main change is submission detection: configurable keywords identify a submission and the MAL account that created the forum reply supplies the username.

Google Form → Google Sheet → MAL Form Monitor → Excel Report

The normal workflow is simple: collect responses, identify qualifying MAL forum replies, match the forum username to the latest Google Form response, evaluate the score, and write the Excel report.

What changed from the previous guide?

The old fixed forum-message format is no longer required. Responders do not need to type a separate Username: line in the forum message. The forum account name is used.

Quick workflow

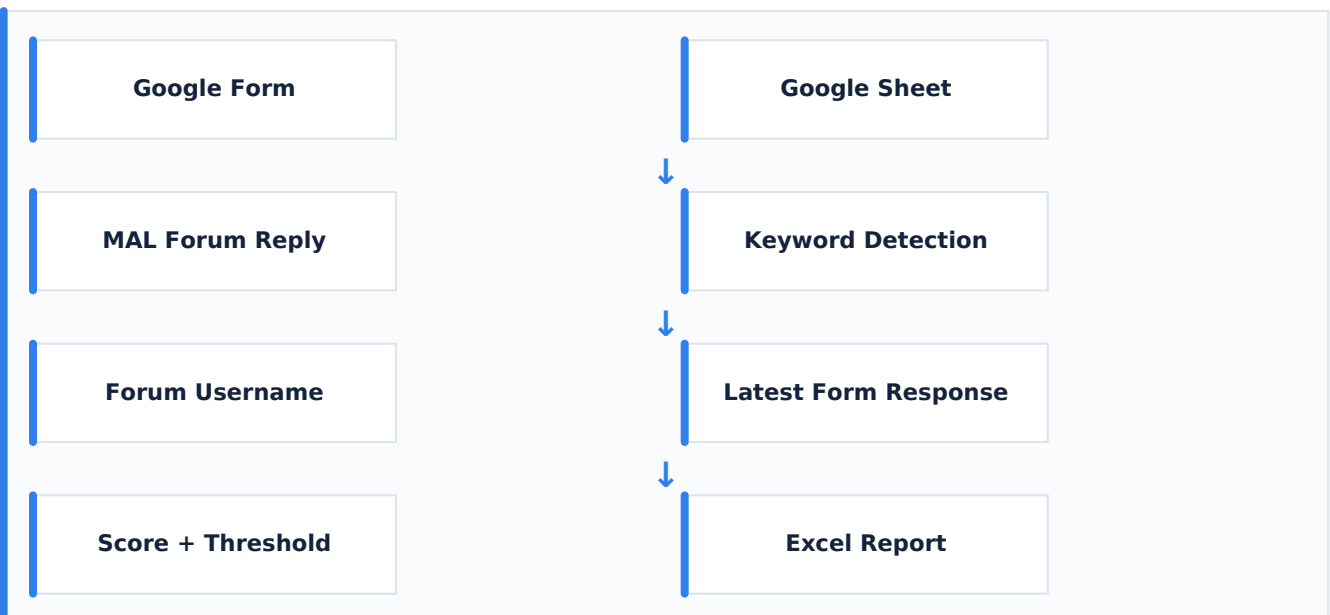


Table of Contents

Click any section below to jump directly to it. The links are built into the PDF.

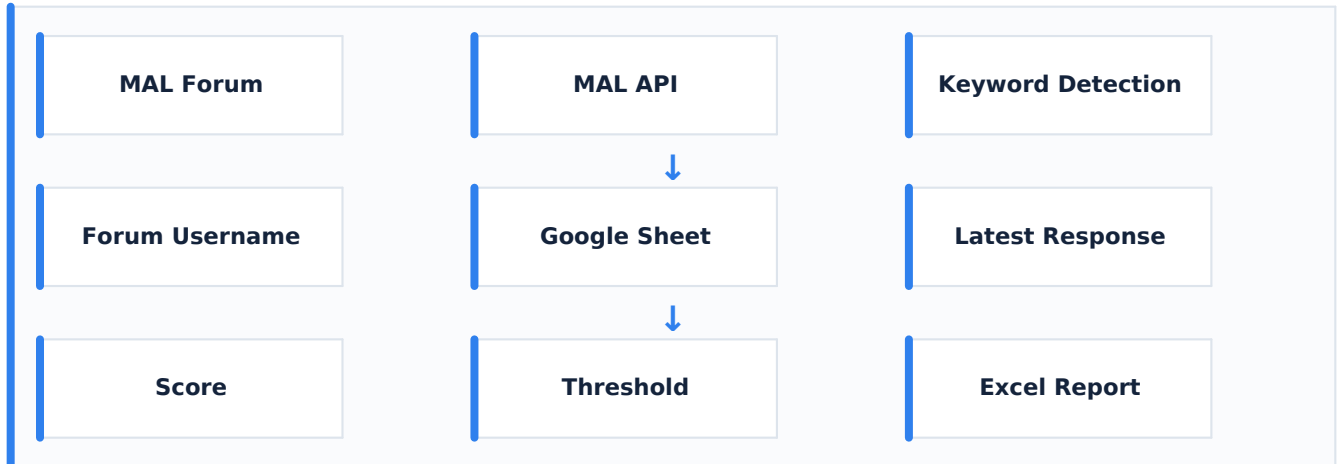
1. [What the program does](#)
2. [What you need before starting](#)
3. [Getting a MyAnimeList API Client ID](#)
4. [Creating the Google Form](#)
5. [Connecting the Form to Google Sheets](#)
6. [Making the Google Sheet readable](#)
7. [Finding the Spreadsheet ID](#)
8. [Configuring the .env file](#)
9. [Submission keyword detection](#)
10. [Running the program](#)
11. [What the program does internally](#)
12. [Understanding the output files](#)
13. [Normal day-to-day workflow](#)
14. [Testing the complete setup](#)
15. [Common problems and fixes](#)
16. [Security and privacy](#)
17. [Important limitations](#)
18. [Final setup checklist](#)

Current workflow at a glance

Google Form → Google Sheet → MAL Forum → MAL API → keyword detection → forum username → latest matching form response → score evaluation → Excel report.

1. What the program does

MAL Form Monitor connects a MyAnimeList forum topic with a Google Form response sheet. When a forum reply contains one of the configured submission keywords, the program treats that reply as a submission event.



The three possible result categories are:

- Passed — a matching form response was found and its score meets the configured threshold.
- Failed — a matching form response was found but the score is below the threshold.
- Missing — a qualifying MAL forum reply was found but no matching Google Form response was found.

Important change

The current program no longer depends on a message such as Submitted the form! followed by Username: The responder can write a normal submission message containing a configured keyword.

2. What you need before starting

Prepare the following before configuring the project:

- A MyAnimeList account.
- A MAL API Client ID.
- A Google account.
- A Google Form.
- A Google Sheet containing the form responses.
- Python installed on the computer.
- The `mal-form-monitor` project.
- The project's required Python packages.

The current project structure is roughly:

```
mal-form-monitor/  
|  
├─ main.py  
├─ config.py  
├─ mal_client.py  
├─ sheets_client.py  
├─ state_manager.py  
├─ report_writer.py  
├─ logger_setup.py  
├─ requirements.txt  
├─ .env  
├─  
└─ data/  
    ├─ results.xlsx  
    ├─ processed_state.json  
    └─ monitor.log
```

You normally do not need to edit the Python files

For routine setup and use, configuration is handled through `.env`. The Python files implement the processing logic.

3. Getting a MyAnimeList API Client ID

The current program communicates with the MAL API using the X-MAL-CLIENT-ID request header. For this version, you provide the Client ID in `.env`.

1 Log into MyAnimeList

Open MyAnimeList and sign in to the account you want to associate with the API application.

2 Open the API configuration area

Look for the developer/API configuration area. MAL can change its interface, so button names and locations may change.

3 Create or select an API application

The interface may ask for application details such as an app name, description, homepage, or redirect information.

4 Copy the Client ID

The value required by this project is the API Client ID.

```
MAL_CLIENT_ID=your_client_id_here
```

Important

The current program uses a Client ID. It does not require you to configure a user OAuth access token for this workflow.

4. Creating the Google Form

Create the Google Form that users will complete. The form can contain the questions appropriate for your project.

1

Create the form

Create a new Google Form and add the questions that determine the user's score.

2

Collect the MAL username

Include a field for the user's MyAnimeList username. This is still essential because the program uses the MAL forum username to find the corresponding form response.

Example form field

MyAnimeList Username:

[_____]

Username matching matters

The username stored in the Google Sheet should correspond to the MAL account name. The current Sheets client normalizes capitalization and surrounding whitespace, but consistent entry is still recommended.

Example:

MAL forum account: Trickedshark881

Google Form answer: Trickedshark881

Result: matching username

5. Connecting the Form to Google Sheets

Google Forms can store responses in a linked Google Sheet. The monitor reads the response data from that sheet.

1**Open the form**

Open the Google Form that will collect the responses.

2**Open Responses**

Go to the form's Responses area.

3**Open the Sheets destination**

Use the Google Sheets option or the equivalent response-destination control shown by the current Google Forms interface.

4**Create or select the spreadsheet**

Use a new spreadsheet or an existing spreadsheet containing the response data.

What the program expects

The response sheet needs usable columns for the username, timestamp, and score. The current Sheets client normalizes column names and searches for the newest matching response.

6. Making the Google Sheet readable

The current project reads the spreadsheet through Google's CSV export URL. It does not sign into your Google account when fetching the sheet.

```
https://docs.google.com/spreadsheets/d/YOUR_SHEET_ID/gviz/tq?tqx=out:csv
```

1 Open the response spreadsheet

Open the Google Sheet linked to the form.

2 Open Share

Use the spreadsheet's sharing controls.

3 Find General access

Change the access setting so the CSV export can be read by the program.

4 Choose Viewer access

The intended permission is read-only access.

General access:
Anyone with the link
Viewer

Do not choose Editor

The program only needs to read the spreadsheet. It does not need permission to edit, delete, or add responses.

Privacy warning

Anyone who obtains the shareable link may be able to access the sheet. Avoid placing passwords, API secrets, private addresses, phone numbers, or other sensitive information in a sheet exposed this way.

7. Finding the Spreadsheet ID

The Spreadsheet ID is the long identifier between /d/ and /edit in the Sheet URL.

```
https://docs.google.com/spreadsheets/d/1ABCxyz1234567890abcdef/edit#gid=0
```

Therefore the Spreadsheet ID is:

```
1ABCxyz1234567890abcdef
```

Copy only the ID

Do not copy the surrounding Google URL. The project needs only the identifier itself.

Example configuration:

```
SHEET_ID=1aBcDeFgHiJkLmNoPqRsTuVwXyZ
```

Do not include:

```
https://docs.google.com/spreadsheets/d/  
/edit#gid=0
```

8. Configuring the .env file

The current program reads its main settings from .env.

A typical configuration looks like:

```
MAL_CLIENT_ID=...
TOPIC_ID=2280005
SHEET_ID=...
TOTAL_QUESTIONS=4
PASSING_THRESHOLD=1
SUBMISSION_KEYWORDS=submitted,submitted,submit
REPORT_FILE=data/results.xlsx
STATE_FILE=data/processed_state.json
LOG_FILE=data/monitor.log
GENERATE_AUDIT_LOG=true
GENERATE_SUMMARY=true
```

The new setting

SUBMISSION_KEYWORDS controls which words cause a forum reply to be treated as a submission.

Other important values:

- TOPIC_ID — the MAL forum topic to monitor.
- SHEET_ID — the Google Spreadsheet containing form responses.
- TOTAL_QUESTIONS — the quiz/question count used by the project configuration.
- PASSING_THRESHOLD — the minimum score required for Passed.
- REPORT_FILE, STATE_FILE, LOG_FILE — output locations.

Windows .env reminder

If Windows hides file extensions, make sure the file is actually named .env, not .env.txt or .env.env.

9. Submission keyword detection

This is the main behavioral change in the current program. The parser no longer requires a fixed forum message format.

Default configuration:

```
SUBMISSION_KEYWORDS=submitted,submitted,submit
```

Matching behavior:

- Keywords are converted to lowercase for comparison.
- The forum message is cleaned and compared case-insensitively.
- Keywords are matched as independent words.
- Additional message text is ignored.
- The MAL forum account that created the post becomes the username used for the Google Sheet lookup.

Valid examples with the default keywords:

Submitted!

Thanks for participating.

I have submitted the form.

SUBMIT

Submitted the form!

Custom keywords

You can change the list in `.env` by separating entries with commas.

```
SUBMISSION_KEYWORDS=submitted,submit,done
```

Important distinction

The program does not extract a username from the forum message. It gets the username from the MAL forum post's `created_by` account and uses that account name for the Sheet lookup.

10. Running the program

Once the configuration is ready, open Command Prompt in the project folder.

1 Check Python

Run the following command:

```
python --version
```

2 Install dependencies

If required, install the packages listed by the project:

```
python -m pip install -r requirements.txt
```

3 Start the monitor

Run the main program:

```
python main.py
```

Expected high-level behavior

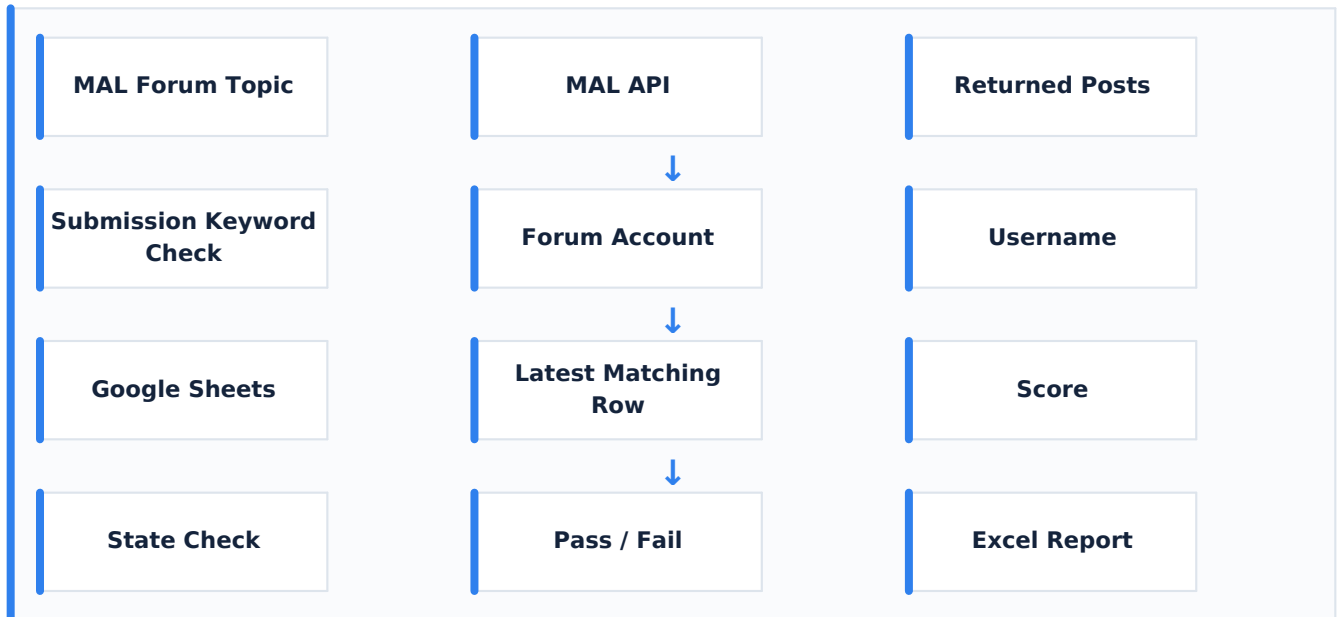
The program fetches forum posts, identifies qualifying submissions, looks up each forum username in Google Sheets, evaluates the latest response, updates the report, and records processing state.

Typical command sequence:

```
cd C:\Users\Admin\Desktop\mal-form-monitor
python --version
python -m pip install -r requirements.txt
python main.py
```

11. What the program does internally

The current processing path can be visualized as:



When multiple Google Form responses exist for the same username:

09:00 → 1/4
10:00 → 2/4
11:00 → 4/4

Selected response:
11:00 → 4/4

The state manager then prevents the same form submission timestamp from being processed repeatedly for that normalized username.

Score evaluation

The score is parsed from the response and compared with `PASSING_THRESHOLD`. A score at or above the threshold is recorded as Passed; a lower score is recorded as Failed.

12. Understanding the output files

The data folder contains the main runtime outputs.

1. results.xlsx

This is the main Excel report. The current report writer can produce Results, Summary, and Audit Log worksheets.

```
data/results.xlsx
```

- Results — Passed, Failed, and Missing records.
- Summary — counts such as Passed, Failed, Missing, and Total Processed.
- Audit Log — reply number, forum user, submitted username, score, result, and form timestamp.

2. processed_state.json

```
data/processed_state.json
```

Stores the processed timestamp for each normalized username so that the same form response is not processed repeatedly.

3. monitor.log

```
data/monitor.log
```

Contains program log messages and is one of the first places to check when troubleshooting.

13. Normal day-to-day workflow

1 Users complete the Google Form

The response is stored in the linked Google Sheet.

2 Users post on the MAL forum

The reply should contain one of the configured submission keywords.

Submitted!

or

I have submitted the form.

3 Run the monitor

Use:

```
python main.py
```

4 Open the report

Open:

```
data/results.xlsx
```

5 Check the result

Review the Passed, Failed, and Missing sections.

No separate username line is required

The responder does not need to type Username: ... in the MAL message. The program uses the MAL account that posted the reply.

14. Testing the complete setup

Before relying on the monitor with real users, test the complete pipeline from end to end.

Test A — Passing response

- Submit the Google Form using a test MAL username.
- Confirm the response appears in Google Sheets.
- Post a MAL forum reply containing a configured keyword.
- Run `python main.py`.
- Check the user in Passed if the score meets the threshold.

Test B — Missing user

- Post a keyword-containing reply from a test account.
- Do not create a matching Google Form response.
- Run the program.
- The user should appear under Missing.

Test C — Duplicate prevention

- Run the program once and allow a form response to be processed.
- Run it again without changing the relevant response.
- The same processed response should not be processed again.

A useful test matrix

Keyword + matching Sheet row + passing score → Passed
Keyword + matching Sheet row + low score → Failed
Keyword + no matching Sheet row → Missing
No keyword → not treated as a submission

15. Common problems and fixes

Problem: A submission is not detected

- Check that the forum reply contains a configured SUBMISSION_KEYWORDS entry.
- Check spelling and capitalization.
- Check that the MAL API actually returned the post.

Problem: The user appears as Missing

- Check that the MAL forum username matches the Google Sheet username.
- Check that the response is actually present in the response sheet.
- Check the response timestamp and whether a newer response exists.

Problem: Google Sheet cannot be read

- Verify SHEET_ID.
- Verify the sheet is accessible to the CSV export request.
- Verify the sheet contains usable username, timestamp, and score columns.

Problem: Python/module error

```
python -m pip install -r requirements.txt
```

Problem: The program sees fewer MAL posts than the website

This is an API/data-visibility issue

The Python program receives forum data from the MAL API rather than scraping the MAL webpage. If the website displays a post but the API does not return that post, the parser never receives it and therefore cannot detect it.

16. Security and privacy

Keep .env private

Do not publish your complete .env file when it contains your real Client ID or other configuration values you do not want to disclose.

- Do not commit private configuration or private runtime data to a public repository.
- Remember that an 'Anyone with the link' Google Sheet can be accessed by people who obtain the link.
- Avoid storing sensitive personal information in the response sheet.
- Use Viewer access because the monitor only needs to read the response data.

A minimal response sheet is easier to reason about:

Timestamp | Username | Score

API visibility is separate from sheet privacy

The MAL forum API determines which forum posts reach the Python program. Google Sheet access determines whether the program can read the response data. These are two separate parts of the pipeline.

17. Important limitations

The current system has several boundaries that are important to understand.

1. MAL API visibility

The program can only process posts returned by the MAL API. A post visible on the MAL website is not automatically guaranteed to be present in the API response.

```
MAL website
↓
Post visible
↓
MAL API
↓
Post returned? — No —> Python cannot process it
|
Yes
↓
Keyword detection
```

2. Keyword detection is not intent understanding

The current parser checks for configured keyword words. It does not determine whether the writer genuinely intended the word as a submission confirmation.

3. Username source

The username used for the Google Sheet lookup comes from the MAL forum account that created the post. It is not extracted from a 'Username:' field inside the message.

Topic 2280005 investigation

The earlier diagnostic showed that the API response for topic 2280005 did not contain every post visible on the website. That distinction should be kept separate from the keyword parser itself.

18. Final setup checklist

Use this checklist before your first real run.

MyAnimeList

- ☐ MAL account available
- ☐ API application created
- ☐ Client ID copied
- ☐ Topic ID is correct

Google Form / Sheets

- ☐ Form is linked to a response spreadsheet
- ☐ MAL username is collected in the form
- ☐ Timestamp and score are available
- ☐ Sheet is readable by the program
- ☐ Spreadsheet ID is correct

.env

- ☐ MAL_CLIENT_ID is correct
- ☐ TOPIC_ID is correct
- ☐ SHEET_ID is correct
- ☐ SUBMISSION_KEYWORDS are correct
- ☐ PASSING_THRESHOLD is correct
- ☐ File is named .env, not .env.txt

Python / runtime

- ☐ python --version works
- ☐ Dependencies are installed
- ☐ python main.py runs
- ☐ data/results.xlsx is generated

Remember the current submission behavior

A responder only needs to post a forum message containing one of the configured submission keywords. The program then uses the MAL account that posted the reply as the username for the Google Sheet lookup.

[↑ Back to Table of Contents](#)